

FORWARD DEPLOYED AI ENGINEERING

Capstone Project

Governance Framework

What must be true before this could speak to a real customer

Risk, fairness, decision logging and incident response. This is the part teams leave until the end, which is precisely why it is usually shallow.

REFERENCE · USE IN WEEK THREE

What governance actually means here

You are building something that will send text to a paying customer without a human reading it first. Governance is the set of arrangements that make that defensible. It is not a document you produce at the end to satisfy a requirement; it is a series of design decisions taken early, which is why this framework asks you to make some of them in week one.

The test of whether your governance is real is simple. Pick a plausible way your system could harm a customer, and ask what in your design would stop it. If the answer describes a property of the model rather than a control you built, you have described a hope.

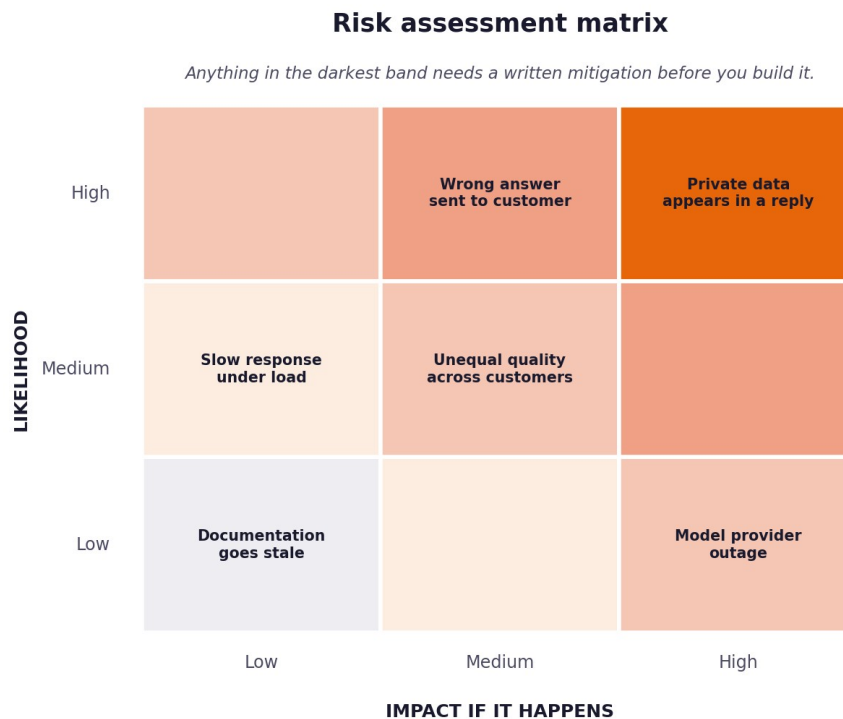


Figure 1. Anything in the darkest band requires a written mitigation before you build it.

1 Decision logging

Every automated decision must be reconstructable months later by someone who was not there. That means logging not only what was decided but why, with enough context that the decision could be audited or challenged.

The minimum record

```
{
  "decision_id": "...",
  "timestamp": "...",
  "ticket_id": "...",
  "stage": "classification | routing | generation | validation",
  "input_summary": "...",
  "model": { "name": "...", "version": "..." },
  "prediction": { "value": "...", "confidence": 0.00 },
  "alternatives": [ { "value": "...", "confidence": 0.00 } ],
  "sources_used": [ { "doc_id": "...", "score": 0.00 } ],
  "threshold_applied": 0.00,
  "action_taken": "auto_respond | escalate | block",
  "reason": "human readable explanation of why this action followed",
  "guardrail_results": { "pii": "pass", "grounding": "pass", "tone": "pass" },
  "prompt_version": "PR-02 v1.3",
  "requirement_ids": ["FR-03", "FR-07"]
}
```

Note the last two fields. Recording which prompt version and which requirement produced a decision is what lets you answer the question that always comes up after a problem: was this behaviour intended?

Coverage check

Count your logged decisions against your processed tickets. They must reconcile exactly. A gap means some path through your system takes a decision without recording it, and that is the path where something will eventually go wrong unobserved.

2 Risk register

Complete this fully. Each risk needs a likelihood, an impact, a specific mitigation and a named person accountable for it. Mitigations phrased as we will be careful are not mitigations.

ID	Risk	Likelihood	Impact	Mitigation in your design	Owner
R-01	The system answers confidently and incorrectly				
R-02	Private data appears in an outbound response				
R-03	A customer's input is treated as an instruction				
R-04	Some customer groups receive worse answers				
R-05	The documentation the system relies on goes stale				
R-06	The model provider becomes unavailable				
R-07	Latency degrades under load				
R-08	Costs rise unexpectedly with volume				

3 Fairness audit

Your system will not treat all customers identically, and you need to know in which direction and by how much. Segment your test set and compare outcomes across segments.

Segment	Tickets in segment	Resolution rate	Quality score	Variation from best	Explanation
Enterprise customers					

Small business customers					
Tickets in fluent English					
Tickets in non-fluent English					
Short tickets					
Long or complex tickets					

The variation you are most likely to find

Systems built on retrieval tend to perform noticeably worse on tickets written in non-fluent English, because the retrieval step depends on the phrasing matching the documentation. This is a real and common fairness problem. Finding it and reporting it honestly is worth far more than a table showing no variation anywhere.

4 Guardrails

A guardrail is a check that runs on every response and can block it. Guardrails that only run in testing are not guardrails; they are tests.

Guardrail	What it checks	What happens when it fires
Private data	Names, addresses, keys, account numbers, anything identifying another customer.	Block and escalate. Never redact and send.
Grounding	Every factual claim traceable to a retrieved passage.	Block and escalate with the unsupported claim flagged.
Instruction integrity	The ticket content has not altered the system's instructions.	Block, escalate and record the input for review.
Tone and scope	The response stays within support scope and does not make commitments.	Block. Commitments about refunds or timelines are not the system's to make.
Confidence floor	The routing threshold was actually applied.	Escalate. A missing confidence score is not a high one.

5 Incident response

Write this so that someone unfamiliar with your system could follow it at two in the morning. That constraint is what forces the useful level of detail.

Step	What to do	Who does it	How long it should take
1. Detect			
2. Contain			
3. Assess			
4. Notify			
5. Remediate			
6. Review			

The kill switch

Every system of this kind needs a way to stop it answering automatically, immediately, without a deployment. State what yours is, who can operate it, and how long it takes to take effect. If you do not have one, that is your first governance finding.

Question	Your answer
What is the mechanism?	

Who is authorised to use it?	
How long until it takes effect?	
What happens to tickets in flight?	
How is it tested?	

6 The declaration

Complete this last. It is the summary an assessor reads first.

Statement	Your position
This system must never ...	
The mechanism that enforces that is ...	
The most likely way it could still cause harm is ...	
We would not deploy this without first ...	